

Trabajo Práctico 1

Regresión e Introducción a la evaluación de modelos

72.75 — Aprendizaje Automático (Machine Learning)
Instituto Tecnológico de Buenos Aires

2026 Q2 · Defensa: 26/08/2026

Resumen

Se implementan modelos de regresión lineal y polinómica para predecir el costo anual de gastos médicos (**charges**) del dataset *Insurance Charges*, evaluados con validación cruzada de $k = 5$ folds. Todo el aparato de evaluación — partición train/test, k -fold, codificación *one-hot*, estandarizado, ecuación normal, expansión polinómica y Lasso por descenso por coordenadas — está implementado desde cero sobre `numpy`, sin `scikit-learn`, tal como pide el punto 2.2 del enunciado.

El modelo elegido para producción es una **regresión lineal de grado 1 sin regularizar**, con once características, que alcanza un **RMSE de test de 4.288,52 dólares** ($R^2 = 0,871$). Es, a la vez, el de **menor error de validación cruzada** entre las 15 configuraciones elegibles y el **más simple** de todo el espacio de búsqueda: no hay que elegir entre esas dos cosas. La regla de 1 error estándar lo confirma igual — es también la más simple de las 7 configuraciones indistinguibles del mínimo —, pero no es lo que decide.

Que el modelo más simple del espacio de búsqueda gane no es casualidad: tres de las once características — `fumador_obeso`, `edad_al_cuadrado` y `bmi_si_fuma`, construidas a partir del análisis exploratorio del punto 1 — capturan de forma explícita la curvatura y las interacciones no aditivas del dataset, y con ellas ni la expansión polinómica ni la regularización agregan nada. Es el resultado central de este trabajo, y se desarrolla en la sección 2.5.

Índice

1. Introducción teórica: separación train / validación / test	2
2. Punto 1 — Limpieza de datos	3
2.1. Distribución de las variables	3
2.2. Variables categóricas	4
2.3. Datos faltantes	4
2.4. Outliers	4
2.5. Las tres poblaciones de charges , y la característica que salió de ahí	5
2.6. Características y escalado	7
3. Puntos 2 y 3 — Implementación	8
3.1. El pipeline, de punta a punta	8
3.2. Cómo se separaron train y test, y por qué así	9
3.3. La regresión polinómica sigue siendo lineal en los parámetros	9
3.4. Regularización L_1	9
4. Punto 4 — Resultados	10
4.1. Dónde está el sobreajuste, y cómo se lo reconoce	11
5. Punto 5 — Las tres respuestas	13
5.1. ¿Qué modelo obtuvo menor error?	13
5.2. ¿Cuál se implementaría en una aplicación real?	14
5.3. ¿Qué RMSE se espera en datos nuevos?	14

5.4. Los once coeficientes del modelo de producción	16
5.5. Sensibilidad al número de folds: ¿y si k no fuera 5?	17
5.6. Dónde vive el error que queda	19
6. Limitaciones	20
7. Guion de la presentación (10 minutos)	22
A. Reproducibilidad	22

1. Introducción teórica: separación train / validación / test

El objetivo de un modelo de regresión no es ajustar bien los datos que ya vio, sino predecir bien datos que **todavía no vio**. Medir el error sobre el propio conjunto de entrenamiento no sirve para eso: el modelo ajusta sus parámetros exactamente para minimizar ese error, así que un RMSE de train bajo puede significar dos cosas radicalmente distintas — que el modelo capturó el patrón real, o que memorizó el ruido particular de esas 1070 filas. Sin un conjunto separado no hay forma de distinguir una cosa de la otra.

Por eso el trabajo separa tres roles, cada uno con un dato distinto y ninguno intercambiable:

- **Entrenamiento (train)**. Ajusta los parámetros del modelo, es decir los coeficientes de la regresión.
- **Validación**. No participa del ajuste; se usa para **elegir entre modelos** — qué grado polinómico, qué valor de λ en Lasso. Es la vara con la que compiten las configuraciones.
- **Test**. Se toca **una única vez**, al final, después de que grado y λ ya están fijados. Estima el error que el modelo va a tener en producción, sobre datos genuinamente nuevos.

La razón por la que el test tiene que quedar intacto hasta el final es sutil pero central: en el momento en que su error se usa para decidir algo — elegir un hiperparámetro, comparar dos arquitecturas, hasta simplemente “mirarlo y probar de nuevo” — deja de ser una medida honesta del error de generalización y pasa a ser, de hecho, un segundo conjunto de validación. El número que reporta ya no dice cuánto va a fallar el modelo en el mundo real, sino cuánto se sobreajustó también al test. En este trabajo el conjunto de test (267 filas) se evaluó siempre *después* de fijar el modelo por validación cruzada, nunca antes: ninguna decisión ajustable —grado del polinomio, λ , características, umbral— se tomó mirando el test. La partición se reutilizó a lo largo del desarrollo, y eso tiene un costo que se reconoce en *Limitaciones*.

Dónde se parte el dataset, y por qué ahí. La separación no ocurre justo antes de entrenar sino **antes del análisis exploratorio**. El motivo es que el análisis exploratorio también decide: de él salen las tres características derivadas de la sección 2.5, y si esas decisiones se toman mirando el dataset completo, el test participó de ellas y deja de estimar el error sobre datos genuinamente nuevos. El orden que sigue el trabajo, entonces, es:

1. limpieza de integridad —nulos y duplicados exactos— sobre el dataset completo;
2. separación train/test;
3. análisis exploratorio y elección de características, **sólo sobre train**.

El primer paso va antes del split porque no toma ninguna decisión estadística: un par de filas idénticas en las 7 columnas es un defecto de carga, y dejarlo pasar rompería la independencia del test antes de que el modelo aprenda nada. Todo lo demás —distribuciones, outliers, correlaciones, el umbral de `fumador_obeso`— se mide exclusivamente sobre las 1070 filas de entrenamiento.

¿Por qué validación cruzada y no un único split? Con un solo split, el resultado depende de qué filas cayeron del lado de validación por azar: con 1070 datos de train, una partición particular puede favorecer o perjudicar a un modelo simplemente por cómo cayeron los outliers o los casos raros. La validación cruzada evita depender de una sola tirada. Se parte el train en 5 bloques de aproximadamente 214 filas cada uno y se repite el ciclo ajustar–validar 5 veces, dejando cada vez un bloque distinto afuera del ajuste. Así **cada dato valida exactamente una vez y entrena en las otras cuatro vueltas**, y el promedio de los 5 errores tiene mucha menos varianza que el error de un único split: la señal que importa —¿esta configuración generaliza mejor que aquella?— deja de estar enmascarada por el ruido de qué le tocó a cada partición. Con un n chico como este esa reducción de varianza es decisiva; es la diferencia entre poder distinguir dos configuraciones o no.

2. Punto 1 — Limpieza de datos

2.1. Distribución de las variables

Antes de decidir nada sobre los datos hay que mirarlos: estadísticas descriptivas e histogramas, juntos, que es lo que permite entender la forma de cada distribución y detectar valores anómalos. La figura 1 lo hace para las siete variables de una vez.

Todo lo que sigue en esta sección está medido sobre las 1070 filas de train, por el motivo de la sección 1: el análisis exploratorio decide qué características entran al modelo, así que no puede mirar el test. La única excepción es el control de integridad de la sección 2.3, que va antes de partir.

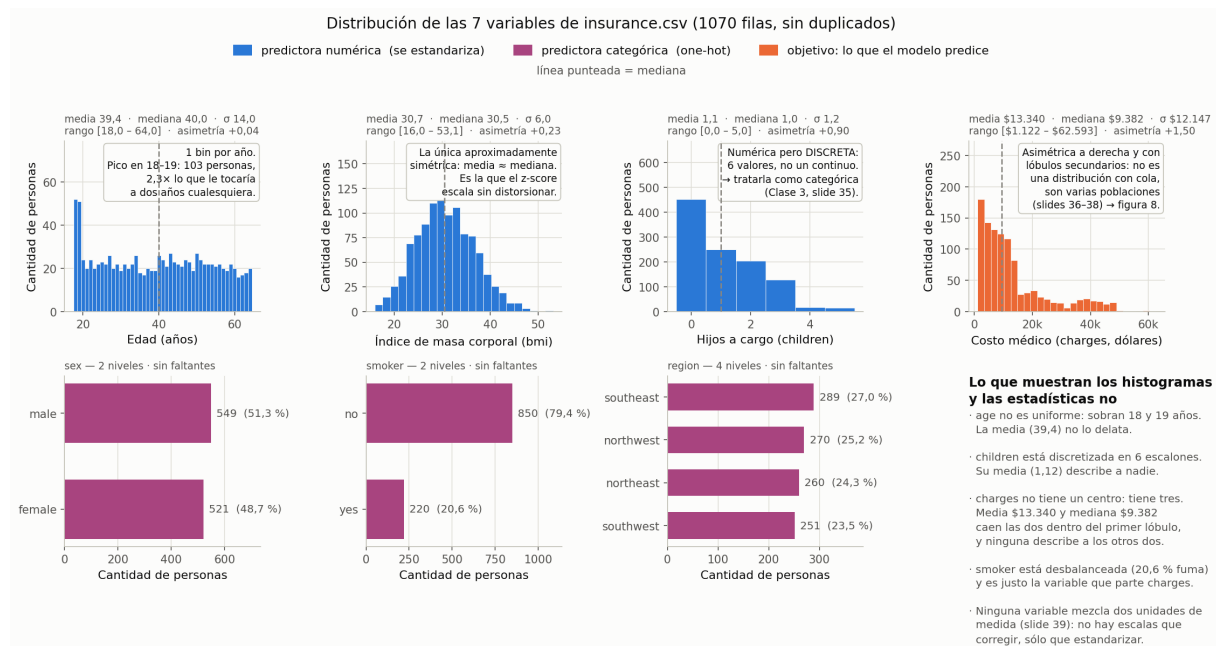


Figura 1: Las siete variables. Arriba las cuatro numéricas, con histograma; abajo las tres categóricas, con frecuencias. El color codifica el rol de cada variable en el modelo. Las numéricas enteras (**age**, **children**) van con un bin por valor; las continuas (**bmi**, **charges**), con el ancho que da la regla de Freedman–Diaconis, que se apoya en el rango intercuartil y por lo tanto no se deja inflar por la cola de **charges**.

Tres cosas aparecen ahí que ninguna tabla de medias muestra, y cada una tiene su análogo literal en los ejemplos del deck:

- **age no es uniforme.** Hay 103 personas de 18 y 19 años, unas 2,3 veces lo que le tocaría a dos años cualesquiera. La media (39,4) no lo delata.

- **children es discreta, no continua:** seis escalones enteros. Una variable así puede tratarse como categórica en vez de numérica; se evaluaron las dos formas y se conserva numérica (sección 2.6).
- **charges no tiene un centro: tiene tres.** Asimetría +1,50 y lóbulos secundarios claros. Es el hallazgo que define el modelo de este trabajo: se desarrolla en la sección siguiente.

Lo que *no* aparece también es resultado: ninguna variable muestra dos escalas superpuestas, así que no hay unidades de medida mezcladas que corregir —sólo escalas distintas que estandarizar, que es otra cosa y se trata más abajo.

2.2. Variables categóricas

El dataset tiene tres variables categóricas: **sex** (binaria), **smoker** (binaria) y **region** (cuatro categorías: *northeast*, *northwest*, *southeast*, *southwest*). Se codificaron con **one-hot** y no con codificación ordinal porque esta última le impone al modelo un orden y una distancia numérica que no existen en los datos: asignar 0, 1, 2, 3 a las regiones le estaría diciendo que *southwest* está más lejos de *northeast* que de *southeast* en algún sentido matemático, cuando la variable es puramente nominal.

Para **region** se generaron 4 dummies pero se **descartó una columna**, que queda como categoría de referencia absorbida en la ordenada al origen. Es la forma estándar de evitar la *dummy variable trap*: si las 4 dummies estuvieran todas presentes sumarían siempre 1, lo cual las hace linealmente dependientes de la columna constante del modelo y vuelve singular la matriz de diseño, de modo que $X^T X$ no se puede invertir para resolver por mínimos cuadrados.

Para las binarias alcanza con **una sola columna** (**sex=male**, **smoker=yes**) en lugar de dos: la segunda categoría es exactamente el complemento de la primera, así que agregarla sería la misma trampa de colinealidad, sólo que con dos categorías en vez de cuatro.

Existen alternativas al one-hot —codificar cada categoría por su frecuencia, o por el promedio de **charges** de esa categoría— y ninguna conviene acá. La primera no aporta nada con cuatro categorías de tamaños parecidos. La segunda usa la variable objetivo para construir la entrada, con lo cual habría que calcularla dentro de cada fold para no filtrar información de validación; con sólo cuatro regiones, ese costo no compra nada frente a tres columnas binarias.

El codificador resultante produce 11 columnas: **age**, **bmi**, **children**, **fumador_obeso**, **edad_al_cuadrado**, **bmi_si_fuma**, **sex=male**, **smoker=yes**, **region=northwest**, **region=southeast**, **region=southwest**.

2.3. Datos faltantes

Este es el único análisis del punto 1 que se hace sobre el dataset completo, por el motivo explicado en la sección 1: detectar nulos y duplicados no requiere ninguna decisión estadística.

El dataset no tiene ningún valor nulo en las 1338×7 celdas originales, así que no hace falta ninguna estrategia de imputación ni de eliminación por faltantes. Sí apareció **una fila duplicada exacta** — los índices 195 y 581 son idénticos en las 7 columnas — que se eliminó, quedando 1337 filas: 1070 de train y 267 de test.

La eliminación se hace **antes del split** train/test, y el motivo no es cosmético: un duplicado exacto repartido entre ambos conjuntos —una copia en train y la otra en test— filtraría información del test hacia el entrenamiento sin que el modelo tuviera que generalizar nada. El error de test quedaría artificialmente bajo para esa fila.

2.4. Outliers

Se comparó la detección por **rango intercuartílico (IQR)** contra **z-score**, y se optó por IQR. La tabla de discrepancia muestra por qué:

Tabla 1: Los dos criterios no coinciden, y la discrepancia se explica.

Variable	IQR detecta	Z-score detecta
charges	115 (10,7 %)	5
children	0	16

En **charges**, el z-score detecta muchísimo menos porque su propia definición depende de la media y el desvío estándar, y **charges** tiene una asimetría de 1,50: los valores extremos —que son justamente los que se querría detectar— arrastran la media hacia arriba e **inflan el desvío estándar**, con lo cual “tres desvíos de la media” deja de ser un umbral exigente. Los propios outliers se disfrazan a sí mismos. El IQR, al basarse en cuartiles, es robusto a esa distorsión.

En **children** pasa lo inverso: el z-score marca 16 valores y el IQR ninguno, porque **children** es un entero acotado entre 0 y 5, y “estar a tres desvíos de la media” no significa lo mismo en una variable discreta de rango chico que en una continua sin cota.

Los outliers se conservan. De los 115 outliers de **charges** detectados por IQR, el **97,4 % son fumadores**, contra apenas 11,3 % de fumadores en el resto de la población. Esa desproporción es la evidencia de que **no son errores de carga, son una subpoblación real**: fumar dispara el costo médico de forma no lineal, y esos 115 casos son la cola derecha esperable de esa subpoblación, no ruido. Eliminarlos le escondería al modelo justo el fenómeno más importante del dataset, que es además el que más le importa acertar a una aseguradora.

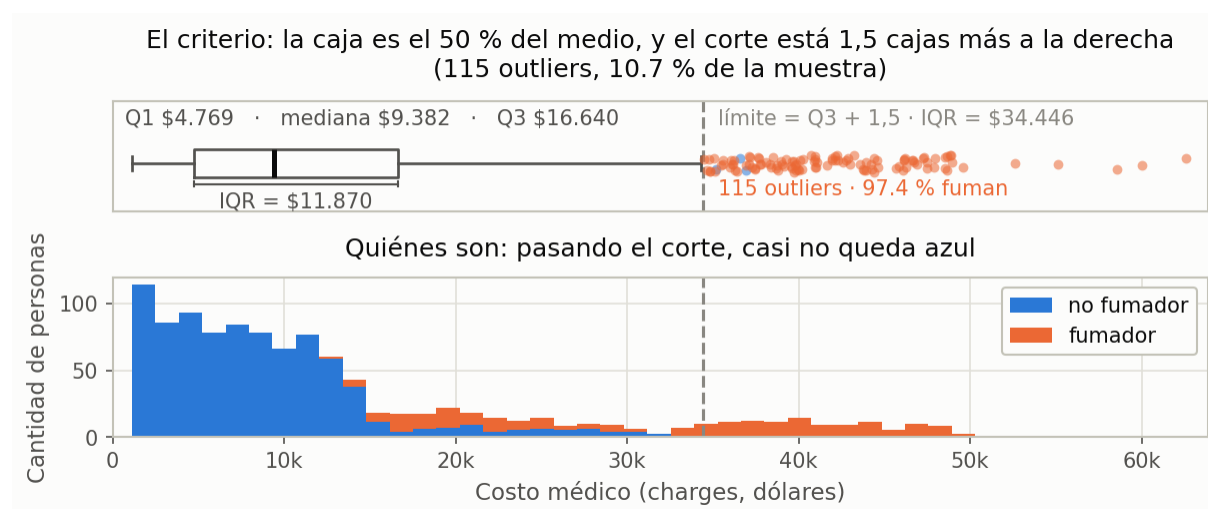


Figura 2: Arriba: de dónde sale el criterio. La caja va de Q_1 a Q_3 —el 50 % central— y su ancho es el IQR; el corte está una caja y media más a la derecha. Abajo: quiénes quedan de ese lado, con la misma distribución apilada por condición de fumador. Los dos paneles comparten el eje horizontal, así que el umbral punteado es una sola vertical que los cruza. La cola larga por encima del corte es *casi enteramente* de fumadores (97,4 %), que es el argumento para conservarla.

2.5. Las tres poblaciones de **charges**, y la característica que salió de ahí

El histograma de **charges** muestra un pico grande y dos jorobas a la derecha. Cuando una variable tiene más de una población hay dos caminos: (a) darle esa información al modelo como variable binaria, o (b) partir el dataset y entrenar un modelo por población. Antes de elegir hay que verificar que las poblaciones existen de verdad, mirando el histograma separado para cada una. La figura 3 es ese paso.

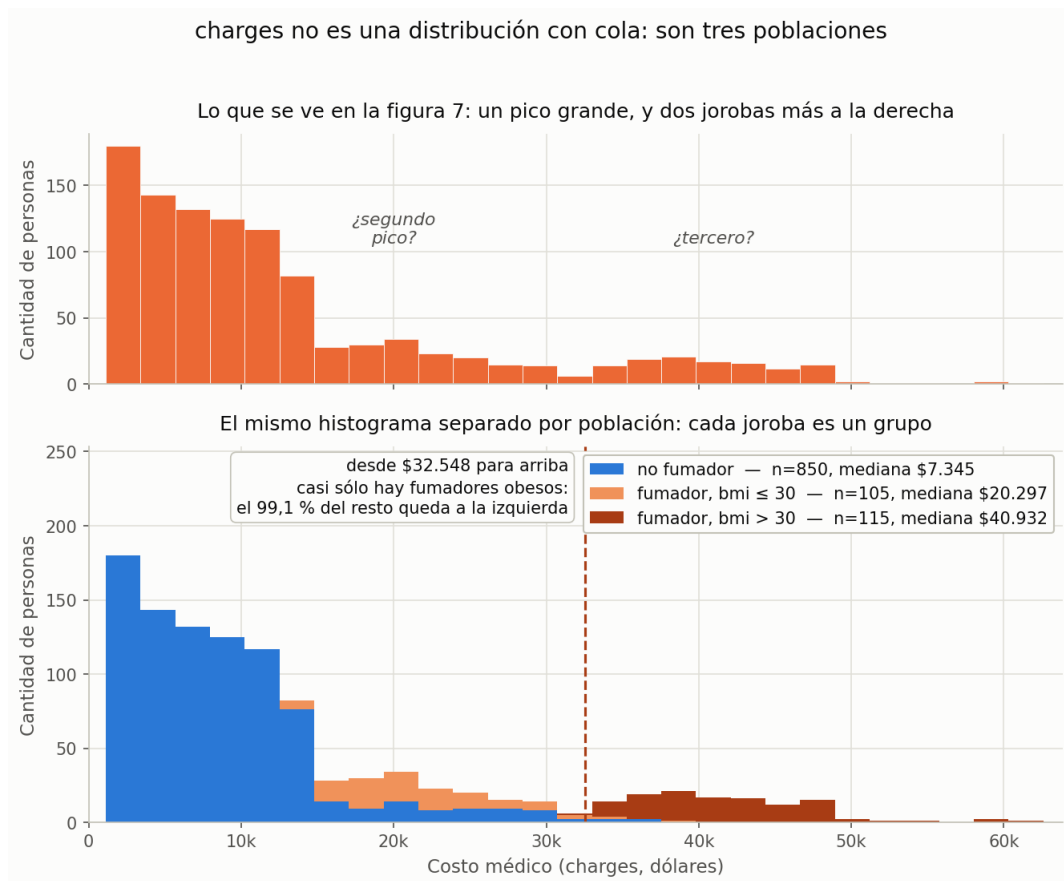


Figura 3: Arriba, `charges` sin separar. Abajo, el mismo histograma con los mismos bins, partido por población. Las medianas son \$7.345, \$20.297 y \$40.932.

Son tres, y no se separan igual de bien. El grupo de fumadores con $\text{bmi} \leq 30$ (105 personas) se solapa con la cola de los no fumadores (850); el de fumadores con $\text{bmi} > 30$ (115) arranca en \$32.548, por encima del 99,1 % de todos los demás. Se eligió el camino (a): con 105 y 115 filas, entrenar un modelo por población dejaría a dos de los tres modelos con muy pocos datos, mientras que una columna binaria le da la misma información al único modelo sin partir nada.

Es un escalón, no una pendiente. La distinción decide qué hacer. Partiendo el `bmi` en tramos finos alrededor de 30, entre los fumadores el costo medio salta de \$24.079 en el tramo $[29, 30)$ a \$38.878 en el $[30, 31)$ —casi quince mil dólares en una unidad de `bmi`—, con pendiente suave a los dos lados del corte; entre los no fumadores el mismo corte casi no mueve nada (\$8.563 contra \$8.089). Si fuera una pendiente, los saltos entre tramos contiguos serían todos parecidos.

Eso importa porque la expansión polinómica de grado 2 *ya* genera el término `bmi*smoker=yes`. Pero un producto con una variable continua sólo puede representar un cambio de pendiente: no puede representar un salto. Por eso se agrega la característica binaria

$$\text{fumador_obeso} = (\text{smoker} = \text{yes}) \wedge (\text{bmi} > 30),$$

El umbral 30 es el de la OMS: es una constante médica, externa a este dataset, y esa procedencia es lo que hace legítimo usarla —no sale de buscar cuál corte minimiza el error. El barrido de umbrales sobre `train` lo confirma igual: el RMSE de validación cae hasta \$4.455 en `bmi > 30` y sube a los dos lados (\$4.776 en 29, \$4.888 en 31).

No es fuga de datos: las dos columnas de las que sale están disponibles al momento de predecir y `charges` no interviene en el cálculo. Y el efecto es de la *interacción*, no de sus partes: agregar sólo `bmi > 30` deja el error prácticamente donde estaba (\$6.039 contra \$6.094 de base), mientras que la interacción completa lo baja a \$4.455.

2.6. Características y escalado

El enunciado pide dos decisiones justificadas, y van por separado.

Qué características se incluyen: las seis originales, sin descartar ninguna, más dos derivadas. El enunciado admite explícitamente que “pueden ser todas”, y en este dataset esa es además la respuesta correcta para las seis variables crudas. A ellas se suman `edad_al_cuadrado` y `bmi_si_fuma`, que la tabla documenta junto con las demás porque son la otra mitad de esta misma decisión: no alcanza con no tirar nada, también hay que agregar lo que la estructura del dataset pide (`fumador_obeso` ya se presentó en la sección 2.5 con la misma lógica). Las tres derivadas se miden sobre train, con validación cruzada de 5 folds promediada sobre 8 particiones: partiendo de las 9 características con `fumador_obeso`, `edad_al_cuadrado` baja el RMSE de validación de \$4.455 a \$4.425, `bmi_si_fuma` a \$4.410, y las dos juntas a \$4.380.

Tabla 2: Decisión de inclusión, variable por variable.

Variable	¿Se incluye?	Por qué
<code>age</code>	Sí	Segunda correlación más alta con <code>charges</code> (0,309) y relación monótona clara. En el modelo de producción queda con coeficiente 45,10.
<code>bmi</code>	Sí	Correlación baja (0,194) pero no descartable: su efecto depende de <code>smoker</code> , y Pearson mide relación lineal, no interacciones. Coeficiente 111,20.
<code>children</code>	Sí	Correlación 0,101, la más débil. Se conserva numérica y no como categórica: con <i>one-hot</i> el RMSE de validación empeora en todos los casos medidos (de \$4.380 a \$4.385 en OLS grado 1, y de \$4.406 a \$4.497 en el mejor Lasso). Coeficiente 815,02.
<code>sex</code>	Sí	Binaria, una columna. El modelo de producción no regulariza, así que no hay penalización que la apague: queda con coeficiente -218,32 (<code>sex=male</code>), de los más chicos en magnitud entre los once, pero no nulo.
<code>smoker</code>	Sí	La variable más informativa del dataset (correlación 0,786).
<code>region</code>	Sí	Tres columnas tras el one-hot. Aporta poco, pero es la única variable geográfica y su costo es mínimo.
<code>edad_al_cuadrado</code>	Sí	Derivada: captura la <i>curvatura</i> del costo con la edad, que un término lineal en <code>age</code> no puede representar. Coeficiente 3749,13, el tercero más grande de los once.
<code>bmi_si_fuma</code>	Sí	Derivada: captura la <i>pendiente</i> del bmi, distinta entre fumadores y no fumadores. Coeficiente 5236,99, el más grande de los once.

Advertencia de interpretabilidad. Agregar `edad_al_cuadrado` y `bmi_si_fuma` tiene un costo. `edad_al_cuadrado` es, literalmente, una función determinista de `age`, y `bmi_si_fuma` lo es de `bmi` dentro del grupo de fumadores: están muy correlacionadas entre sí. Por eso los coeficientes de `age` (45,10), `bmi` (111,20) y `smoker=yes` (1161,22) quedan chicos: las derivadas absorbieron buena parte del efecto que esas variables cargarían solas. Eso no significa que `age` o `bmi` dejaron de importar — significa que, dentro de cada grupo (`age` con `edad_al_cuadrado`; `bmi`, `smoker=yes` y `fumador_obeso` con `bmi_si_fuma`), los coeficientes individuales ya no son interpretables por separado: sólo tiene sentido leerlos como grupo. Es un costo real de interpretabilidad a cambio

de mejor ajuste.

El criterio general es que **no se descarta ninguna variable a mano**. Con 1070 filas de train y sólo 11 columnas tras codificar no hay problema de dimensionalidad que lo justifique: descartar por correlación baja sería tirar información a cambio de nada. Descartar por correlación es además un criterio que sólo mira relaciones lineales de a una variable por vez, y este dataset tiene justamente el caso contrario: **bmi** correlaciona 0,194 con **charges** y sin embargo es, en interacción con **smoker**, la fuente del efecto más grande del modelo. Cuando hace falta seleccionar —en los grados altos, donde la expansión polinómica genera cientos de columnas— la selección se delega a la penalización L_1 , que la hace **dentro de cada fold** y por lo tanto sin fuga.

El escalado. Las variables numéricas se **estandarizan** (z -score: media 0, desvío 1) antes de ajustar cualquier modelo con regularización, porque Lasso penaliza la magnitud de los coeficientes: si las variables están en escalas distintas —**age** en años, **bmi** en kg/m^2 , **children** en unidades enteras chicas— la penalización castiga más a los coeficientes de las variables que naturalmente necesitan valores grandes, sin que eso tenga que ver con su importancia real.

Se eligió z -score y no un reescalado min-max al rango $[0, 1]$ justamente porque este trabajo **conserva** los 115 outliers de **charges**: min-max fija los extremos de la escala en el mínimo y el máximo observados, así que un solo valor extremo comprime a todos los demás contra un costado del rango. El z -score usa media y desvío, que un puñado de valores altos mueve mucho menos.

Para el modelo de producción —lineal, sin regularizar— estandarizar no cambia las predicciones: es una transformación afín de las columnas y mínimos cuadrados la absorbe en los coeficientes. Sí cambia cómo se *leen* esos coeficientes, y por eso la tabla 7 es interpretable: al estar todas las características en desvíos, sus magnitudes son comparables entre sí.

El punto crítico es **cuándo** se calculan la media y el desvío: **siempre sobre el fold de entrenamiento únicamente**, nunca incluyendo el fold de validación. Es la misma lógica que con el test. Si se calcularan sobre todos los datos, la validación dejaría de ser una medición honesta, porque el escalado ya habría “visto” esos datos antes de que el modelo intentara predecirlos.

3. Puntos 2 y 3 — Implementación

3.1. El pipeline, de punta a punta

1. `quitar_duplicados`: $1338 \rightarrow 1337$ filas.
2. `separar_train_test` con semilla 42: 1070 filas de train, 267 de test (80/20). **Es el punto en el que se congela el test**: el análisis exploratorio que eligió las características del paso siguiente corrió únicamente sobre estas 1070 filas (`src.datos.cargar_train`).
3. `agregar_derivadas`: agrega tres columnas —`fumador_obeso`, `edad_al_cuadrado` y `bmi_si_fuma`—. En el código se aplica al DataFrame entero de una sola vez, antes de indexar train y test, y eso es indistinto: son funciones fijas fila a fila de `age`, `bmi` y `smoker`, no aprenden ningún parámetro de la muestra y ninguna usa **charges**, de modo que aplicarlas antes o después de partir da exactamente las mismas columnas. Lo que sí tenía que decidirse sobre train —y se decidió sobre train— es *cuáles* tres columnas agregar y con qué umbral.
4. `CodificadorCategoricas`, ajustado **sólo con train**: produce las 11 columnas descriptas arriba.
5. `k_fold`: partición propia en 5 folds sobre el train.
6. Dentro de cada fold: `Estandarizador` (ajustado sólo con el fold de entrenamiento) $\rightarrow$ `expandir_polinomica` $\rightarrow$ `Estandarizador` otra vez $\rightarrow$ ajuste del modelo.
7. Modelos: `RegresionLineal` (mínimos cuadrados vía descomposición en valores singulares) y `Lasso` (descenso por coordenadas con actualización incremental del residuo).

3.2. Cómo se separaron train y test, y por qué así

El procedimiento concreto es: se genera una **permutación aleatoria** de los índices $0 \dots 1336$ y se corta, mandando las últimas $\text{round}(1337 \times 0,2) = 267$ posiciones a test y las 1070 restantes a train. Tres decisiones dentro de eso:

- **Se baraja antes de cortar.** Tomar las últimas 267 filas del archivo tal como viene sería válido sólo si el orden fuera aleatorio, y eso no se puede dar por sentado: los CSV suelen venir ordenados por algún criterio. Barajar rompe cualquier estructura latente en el orden.
- **La semilla es fija (42).** Sin semilla fija ningún número de este informe sería reproducible. También evita una trampa sutil: si la semilla variara, sería posible correr varias veces y quedarse con la partición que da mejores números, que es una forma encubierta de ajustar contra el test.
- **No se estratifica, y el motivo se midió.** El argumento fácil sería que **charges** es continuo y por lo tanto no hay clases que preservar, pero el análisis exploratorio lo desmiente: **charges** no es una distribución continua unimodal sino *tres poblaciones casi disjuntas* (figura 3), y los cinco folds reparten la tercera de forma desigual —entre 17 y 31 personas, del 7,9 % al 14,5 %—, con correlación +0,62 entre esa proporción y el **charges** medio del fold. Hay clases de facto, y se reparten mal.

Lo que ocurre es que estratificar por ellas *no cambia nada*, y eso se midió sobre train: promediando sobre ocho particiones, el desvío del RMSE entre folds es 593 con folds aleatorios y 591 con folds estratificados —dos dólares, cuando entre dos particiones del mismo método la diferencia va de 410 a 715—. Lo que mueve el error de un fold no es cuántos fumadores obesos le tocaron sino cuáles: dentro de ese grupo los costos van de \$32.548 a \$63.770, así que igualar los conteos deja intacta la varianza que importa. La estratificación controla justo la dimensión que no era el problema.

- **Sí sigue valiendo el resto.** No hay estructura temporal ni de grupos —cada fila es una persona distinta— así que el supuesto de independencia que justifica el barajado se cumple.

3.3. La regresión polinómica sigue siendo lineal en los parámetros

Elevar **age** al cuadrado o multiplicar **bmi** por **smoker=yes** no cambia la naturaleza del modelo: sigue siendo una combinación lineal de columnas de la matriz de diseño, sólo que esas columnas ahora son transformaciones no lineales de las variables originales. Lo que se ajusta por mínimos cuadrados es exactamente el mismo problema que con grado 1; lo único que cambia es qué características se le presentan al modelo.

3.4. Regularización L_1

El objetivo que minimiza el Lasso implementado es

$$J(w) = \frac{1}{2n} \|y - Xw - b\|_2^2 + \lambda \|w\|_1, \quad (1)$$

resuelto por descenso por coordenadas con *soft-thresholding*. Se usa L_1 y no L_2 (Ridge) porque acá la penalización tiene que hacer dos trabajos: controlar la magnitud de los coeficientes y elegir características. En grado 4 la expansión genera 1364 columnas y L_2 las dejaría a todas vivas, apenas encogidas; L_1 lleva la mayoría exactamente a cero —17 sobrevivientes en promedio con $\lambda = 310,83$ — y devuelve un modelo que se puede leer. El intercepto b no se penaliza: se obtiene centrando X e y y recuperándolo como $b = \bar{y} - \bar{x}^T w$. Penalizarlo encogería la predicción media hacia cero, que no es lo que la regularización busca.

La grilla de λ se construye **relativa a $\lambda_{\text{máx}}$** , el menor valor que anula todos los coeficientes:

$$\lambda_{\text{máx}} = \max_j \frac{|x_j^T (y - \bar{y})|}{n}. \quad (2)$$

Ese máximo se calcula sobre el train completo, y da 10360,94 —el mismo valor para los grados 2, 3 y 4—. Una grilla relativa es interpretable y comparable entre grados; una absoluta elegida a ojo, no.

4. Punto 4 — Resultados

Tabla 3: Validación cruzada de 5 folds sobre las 1070 filas de entrenamiento. Regresión lineal sin regularizar, por grado del polinomio. Tabla generada por `src/tablas.py` a partir de `resultados/cv_lineal.csv`.

Grado	RMSE train	RMSE validación	Características
1	4351,6 ± 93,2	4413,4 ± 367,0	11
2	4227,2 ± 104,7	4516,7 ± 380,0	77
3	3931,1 ± 117,1	6757,5 ± 857,0	363
4	3379,1 ± 105,3	87916,9 ± 47000,1	1364

Tabla 4: Lasso, mismo esquema de validación cruzada. Las configuraciones que no convergieron quedan excluidas de la selección. Tabla generada por `src/tablas.py` a partir de `resultados/cv_lasso.csv`.

Grado	λ	RMSE train	RMSE validación	Coefs. $\neq 0$
2	3108,28	6369,5	6397,0	4,4
2	1036,09	4711,7	4730,3	5,8
2	310,83	4408,7	4434,0	9,4
2	103,61	4330,4	4422,9	22,2
2	31,08	4288,7	4459,2	37,4
3	3108,28	6363,7	6390,7	4,2
3	1036,09	4711,0	4730,1	5,4
3	310,83	4389,3	4431,4	15,6
3	103,61	4260,7	4464,4	48,8
3	31,08	4152,0	4621,8	96,0
4	3108,28	6363,4	6394,4	4,2
4	1036,09	4709,9	4732,4	5,6
4	310,83	4378,6	4433,7	17,0
4	103,61	4183,2	4531,6	74,2
4	31,08	3942,5	4938,2	166,6

4.1. Dónde está el sobreajuste, y cómo se lo reconoce

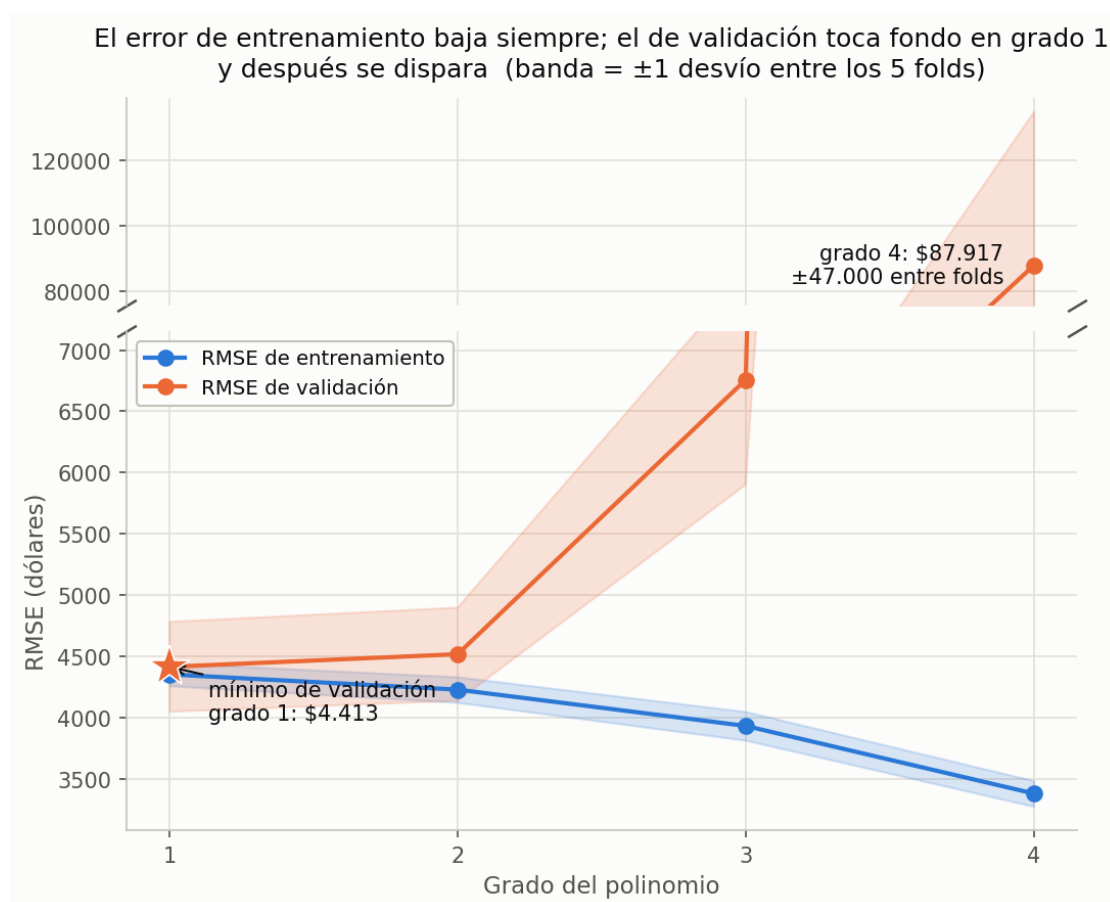


Figura 4: El error de entrenamiento baja monótonamente con el grado; el de validación toca fondo en grado 1 y después sube, hasta dispararse en grado 4. El eje está *partido*: el grado 4 se va a 87.916,9 dólares y en una escala única aplastaría a los otros tres hasta volverlos ilegibles. La banda es ± 1 desvío entre folds.

En regresión lineal sin regularizar el sobreajuste empieza ya en grado 2 y se vuelve catastrófico en grado 4. El error de **train** baja monótonamente (4351,6 $\rightarrow$ 4227,2 $\rightarrow$ 3931,1 $\rightarrow$ 3379,1 dólares), porque más características siempre le dan al modelo más grados de libertad para ajustar lo que ya vio. El de **validación** hace exactamente lo contrario desde el arranque: toca su mínimo en grado 1 (4413,4) y sube en cada grado siguiente, hasta 87916,9 en grado 4 —**19,9 veces** el del grado 1—.

Por qué el grado 4 se dispara así. Hay una forma rápida de anticiparlo, antes de mirar ningún error: contar parámetros contra datos. Una regla práctica pide del orden de diez ejemplos por parámetro, y cada fold entrena con 856 filas.

Tabla 5: Cuántos datos pediría cada grado, contra los 856 que hay por fold de entrenamiento.

Grado	Parámetros	Ejemplos que pediría (10 $\times$)	¿Alcanzan las 856 filas?
1	11	110	sí
2	77	770	justo
3	363	3.630	no
4	1364	13.640	no, y además $p > n$

La tabla predice el orden exacto de la columna de validación de la tabla 3, y el grado 4 es el caso extremo: con más columnas que filas el problema **deja de estar determinado**. El diagnóstico de rango que corre `src/experimentos.py` lo confirma: de las 1364 columnas, el rango efectivo es 436, o sea que el **68,0 %** son combinación lineal exacta de las otras (las causas están en *Limitaciones*; una de ellas es que a partir de grado 2 la expansión polinómica vuelve a generar `edad_al_cuadrado` y `bmi_si_fuma`, que ya están entre las once). En grado 1, el de producción, no hay duplicación: cada característica aparece una sola vez y las once columnas son de rango completo. Con 856 filas por fold, el ajuste de mínimos cuadrados sobre el espacio de grado 4 es una extrapolación, y lo que predice fuera del fold no tiene por qué parecerse a nada.

El indicador más claro de sobreajuste no es sólo que el RMSE de validación suba, sino que la **brecha** entre train y validación crezca sistemáticamente: de **61,8 dólares** en grado 1 a **84.537,9 dólares** en grado 4. Un modelo que generaliza bien tiene un error de validación parecido al de train; una brecha que se ensancha es la firma directa de que el modelo está aprendiendo particularidades del fold de entrenamiento que no se sostienen fuera.

El segundo indicador, más sutil pero igual de contundente, es la **estabilidad entre folds**: el desvío del RMSE de validación pasa de $\pm 367,0$ en grado 1 a $\pm 47,000,1$ en grado 4, **128 veces más grande**. Eso dice algo distinto de “el modelo es peor en promedio”: dice que el modelo de grado 4 es **inestable**, que su desempeño cambia drásticamente según qué filas caen en cada partición.

Una observación sobre el desvío en grado 1. Los $\pm 367,0$ del modelo elegido no son chicos, y eso no es un defecto: es la contracara del hallazgo. Al capturar el escalón de `fumador_obeso`, el modelo elimina el error sistemático que dominaría sin esa columna, y lo que queda es el error idiosincrásico de los casos extremos —cuya repartición entre folds es justamente lo que varía de un fold a otro—. Es el mismo fenómeno que explica por qué estratificar no ayuda.

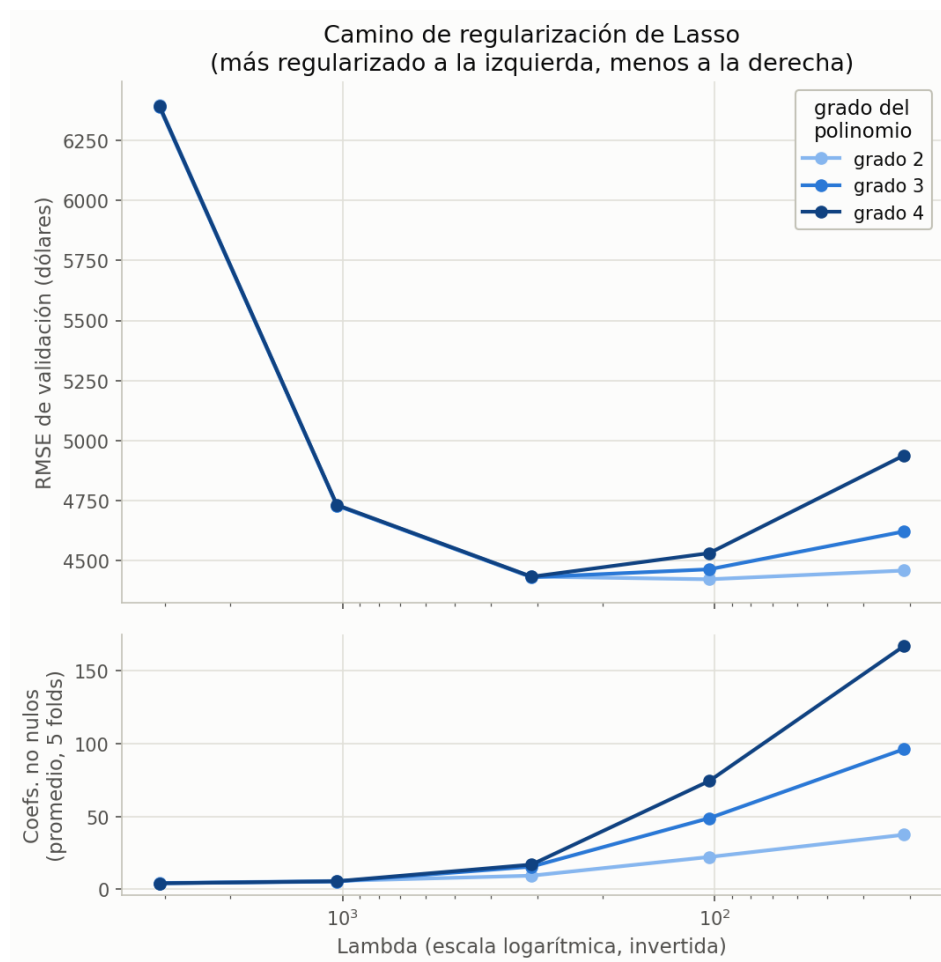


Figura 5: Camino de regularización. Al aflojar λ entran más características (panel inferior) y el error primero baja y después vuelve a subir. El tono codifica el grado del polinomio: más oscuro, grado más alto.

Lasso corrige el problema en buena medida: el mejor λ de grado 4 que efectivamente converge ($\lambda = 310,83$) lleva el RMSE de validación de 87916,9 a 4433,7 dólares —prácticamente las mismas 19,9 veces de distancia que separan al grado 4 sin regularizar del grado 1, pero recorridas en el sentido contrario— dejando vivos 17,0 de los 1364 coeficientes en promedio. Que la penalización L_1 recupere de un espacio degenerado un error comparable al del modelo lineal simple es exactamente lo que la regularización promete, y acá se ve en su forma más extrema.

Lo que *no* logra es superarlo: el mejor Lasso de toda la tabla (grado 2, $\lambda = 103,61$) queda en 4422,92, apenas nueve dólares por encima del lineal de grado 1, con 22,2 características vivas en promedio en lugar de 11. Sobre esa comparación decide el punto siguiente.

5. Punto 5 — Las tres respuestas

5.1. ¿Qué modelo obtuvo menor error?

El de menor RMSE de validación cruzada es, directamente, **la regresión lineal de grado 1 sin regularizar**: RMSE de validación = $4413,45 \pm 366,97$, con las once características de grado 1, todas vivas. El mejor Lasso de toda la búsqueda (grado 2, $\lambda = 103,61$) queda apenas nueve dólares por encima, en 4422,92, con 22,2 coeficientes vivos en promedio.

5.2. ¿Cuál se implementaría en una aplicación real?

El mismo modelo que ganó la sección anterior, y esta vez la pregunta se responde sola: no hay que elegir entre el de menor error y el más simple porque son la misma configuración.

De todos modos vale correr el procedimiento completo, para no decidir por comodidad. Aplicando la **regla de 1 error estándar** sobre el ganador,

$$ES = \frac{366,97}{\sqrt{5}} = 164,11, \quad \text{umbral} = 4413,45 + 164,11 = 4577,56, \quad (3)$$

de las 19 configuraciones corridas, **cuatro** no convergieron y quedan fuera de la comparación: Lasso de grado 3 con $\lambda = 310,83$, $\lambda = 103,61$ y $\lambda = 31,08$, y Lasso de grado 4 con $\lambda = 31,08$. Sobre las 15 configuraciones elegibles que quedan, **7 caen por debajo del umbral**. Esas 7 son, para los datos disponibles, **indistinguibles entre sí**: cuál aparece primera en la tabla depende del azar de cómo cayó la partición en 5 folds, no de una diferencia real de capacidad predictiva.

Frente a un empate estadístico, lo habitual sería resolver por la configuración **más simple** del grupo empatado. Ese paso, esta vez, es innecesario: la más simple de las 7 —once características, grado 1, sin regularizar, sin ningún hiperparámetro que ajustar, y por lejos la de menos columnas en la matriz de diseño— **ya es** la que tiene el menor error de validación cruzada de toda la búsqueda. La regla de 1 ES no decide nada acá: sólo **confirma** lo que el mínimo ya decía. Y como el ganador de la CV y el modelo de producción son el mismo modelo, el costo de la simplicidad es **0,00 dólares**: no hay nada que sacrificar a cambio de un modelo más simple.

Vale subrayar lo que eso significa, porque es el resultado central del trabajo: *con las características correctas, ni la expansión polinómica ni la regularización compran nada*. Las dos existen para darle al modelo la flexibilidad de representar relaciones que las variables crudas no representan; una vez que `fumador_obeso`, `edad_al_cuadrado` y `bmi_si_fuma` las representan de forma explícita, la flexibilidad extra sólo agrega varianza. La evidencia directa está en el punto 1: sin esas columnas derivadas el modelo lineal de grado 1 *sí* se queda corto —\$6.094 de RMSE de validación contra los \$4.380 que alcanza con ellas— y hace falta subir a grado 2 con Lasso para acercarse.

Tabla 6: Resultado sobre el conjunto de test: las 267 filas que no participaron de ninguna decisión. Se reportan las dos métricas juntas: el RMSE en dólares, que es el que pide el enunciado, y R^2 , que lo dimensiona contra la variabilidad que `charges` tiene de por sí. El baseline es la referencia mínima: predecir siempre la media de train.

Modelo	Caract.	RMSE test	R^2 test
Producción (lineal grado 1, sin regularizar)	11	4.288,52	0,871
Baseline: predecir siempre la media	—	11.963,43	0,000

El modelo reduce el error del baseline a algo más de un tercio y explica el **87,1 %** de la varianza de `charges` en datos que nunca vio. Vale subrayar de dónde salió esa mejora: no de agregar grados de libertad al polinomio ni de ajustar un hiperparámetro de regularización — las dos cosas empeoraron el error de validación— sino de las tres características derivadas que salieron del análisis exploratorio del punto 1.

5.3. ¿Qué RMSE se espera en datos nuevos?

4.288,52 dólares — el RMSE de **test** del modelo de producción, no su RMSE de validación (4.413,45).

Es importante entender por qué el número de validación está sesgado a la baja y no se puede prometer. Esa configuración se eligió por tener, directamente, el RMSE de validación **más bajo**

de las 15 estimaciones elegibles (de 19 corridas; cuatro quedaron fuera por no converger). Cada una de esas estimaciones tiene ruido de muestreo, porque es un promedio sobre sólo 5 folds. Cuando se elige el **mínimo** de un conjunto de estimaciones ruidosas, ese mínimo queda sesgado hacia abajo *aunque cada estimación individual sea insesgada*: es más probable que el “ganador” lo sea en parte por haber tenido una racha favorable de ruido, no sólo por ser mejor de verdad. Acá ese argumento aplica de lleno y sin salvedades —no hay un modelo elegido por simplicidad y otro que ganó la CV: hay uno solo, y es exactamente el mínimo de la tabla—. El test, al no haber participado de ninguna decisión, no tiene ese sesgo.

Y sin embargo el test dio *mejor* que la validación: 4.288,52 contra 4.413,45, una diferencia de −124,93 dólares. Eso no contradice el argumento anterior: el sesgo de selección predice que el test tienda a ser peor *en promedio*, no que lo sea en esta partición puntual, y 124,93 dólares es chico frente al $\pm 366,97$ de ruido que ya separaba a los folds entre sí durante la validación. Con una sola partición de test no hay forma de saber si el ruido jugó a favor esta vez o si el sesgo es de verdad más chico de lo que la teoría sugiere.

Dos advertencias que corresponde dar junto con ese número.

- El conjunto de test tiene **267 filas**. El 4.288,52 es en sí mismo una estimación con su propia incertidumbre de muestreo, que este trabajo no cuantificó con un intervalo de confianza. No hay que leerlo con cuatro decimales.
- El RMSE **promedia sobre una población heterogénea**. El error no se reparte parejo: se concentra en los fumadores, que son los casos más caros y los que más le importa acertar a una aseguradora. Prometer un único RMSE global oculta esa asimetría.

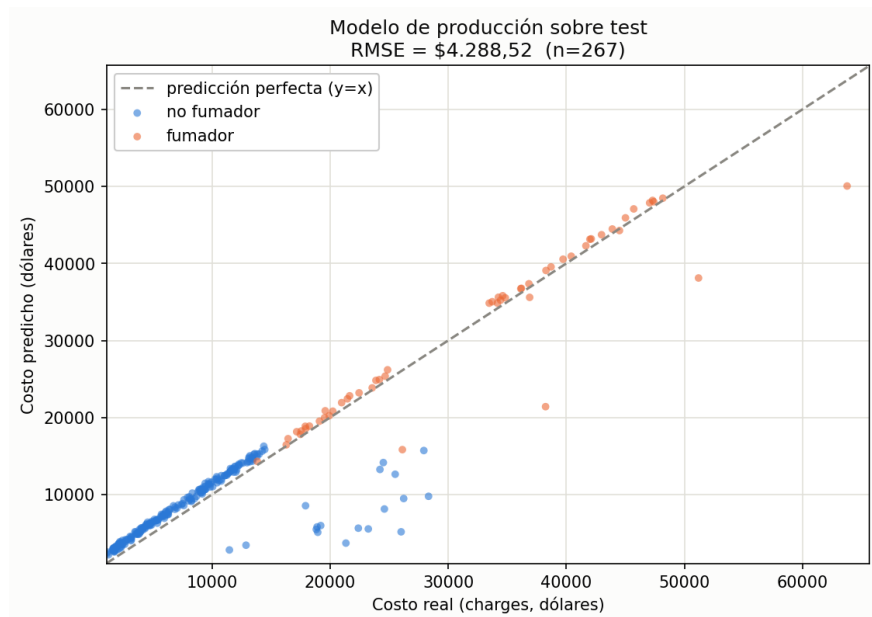


Figura 6: Predicho contra real sobre el conjunto de test. El modelo acierta con precisión en la mayoría barata (nube azul sobre la diagonal) y se dispersa en los casos caros, que son casi todos fumadores: es la asimetría del error que la segunda advertencia menciona.

5.4. Los once coeficientes del modelo de producción

Tabla 7: El modelo entero. Son las once características de grado 1, sin penalización: no hay selección que reportar porque no hay nada de más que apagar. Los coeficientes están sobre características estandarizadas, así que sus magnitudes son comparables entre sí —con una salvedad importante: `age` y `edad_al_cuadrado` son función determinista una de la otra, y lo mismo `bmi`, `smoker=yes` y `bmi_si_fuma` entre sí. Dentro de cada uno de esos grupos colineales el coeficiente individual no se interpreta solo: hay que leer el grupo completo (ver *Advertencia de interpretabilidad*, más arriba).

Característica	Coeficiente
<code>bmi_si_fuma</code>	5236,99
<code>fumador_obeso</code>	4891,83
<code>edad_al_cuadrado</code>	3749,13
<code>smoker=yes</code>	1161,22
<code>children</code>	815,02
<code>region=southwest</code>	−498,36
<code>region=southeast</code>	−424,86
<code>sex=male</code>	−218,32
<code>region=northwest</code>	−142,53
<code>bmi</code>	111,20
<code>age</code>	45,10

El hallazgo. Los tres coeficientes más grandes del modelo son, en orden, las tres características derivadas: `bmi_si_fuma` (5236,99), `fumador_obeso` (4891,83) y `edad_al_cuadrado` (3749,13), las tres por encima de `smoker=yes` (1161,22), que sobre las variables crudas es el término dominante. El modelo dice, leído en orden: la pendiente del `bmi` entre fumadores es lo que más pesa; ser fumador *y* obeso suma otro tanto encima; la curvatura de la edad importa casi tanto como las dos anteriores; y recién después aparece el efecto de fumar por sí solo, ya reducido porque buena parte de lo que hacía antes —la pendiente distinta según el `bmi`— ahora lo explica `bmi_si_fuma` de forma directa. Es el mismo fenómeno que la advertencia de interpretabilidad ya anticipaba: las características derivadas no compiten con las originales por espacio en el modelo, les *absorben* el efecto.

Vale la pena contrastarlo con lo que hace el Lasso cuando no se le da `fumador_obeso` y tiene que arreglarse con la expansión polinómica: deja vivo el término `bmi*smoker=yes` entre los de mayor magnitud, por encima de `bmi` sola. *El Lasso encuentra la interacción por su cuenta*, sin que nadie se la indique, y eso es una buena demostración de para qué sirve L_1 —pero la encuentra en la única forma que su espacio de características le permite, como un cambio de pendiente. El análisis exploratorio mostró que la forma correcta era un escalón, y darle esa columna directamente al modelo resultó valer más que toda la maquinaria que la aproximaba.

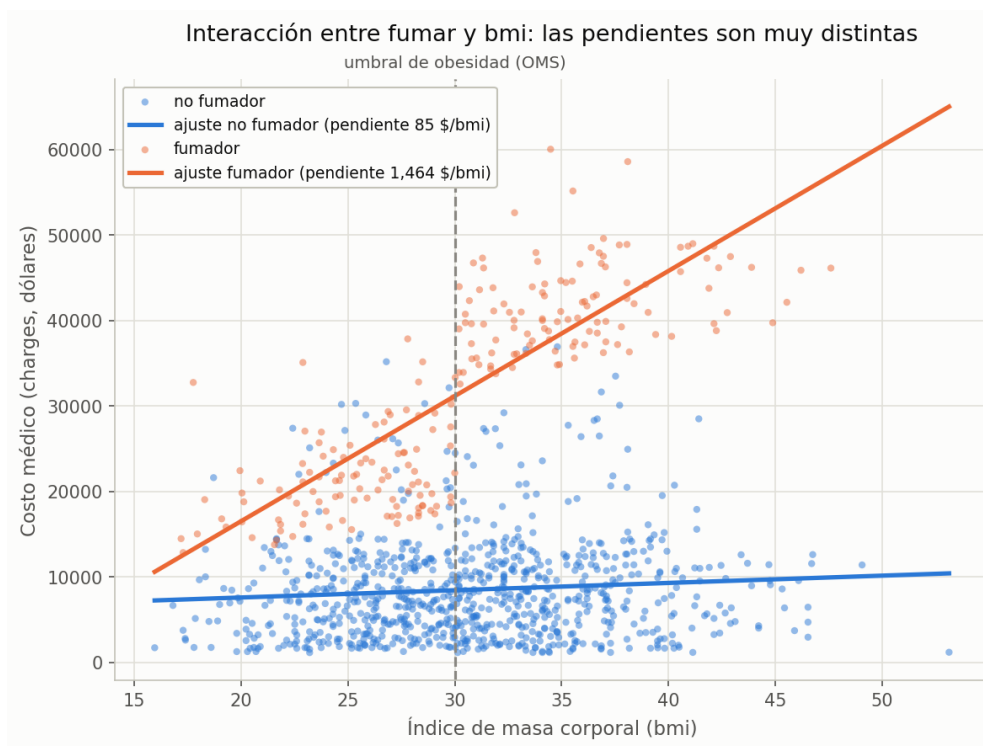


Figura 7: La interacción, cuantificada. Ajustando una recta por grupo, la pendiente es de **83 dólares por unidad de BMI** entre los no fumadores y de **1473** entre los fumadores: un factor de **17,7**. Un modelo puramente aditivo sobre las variables crudas no puede representar esa diferencia; `fumador_obeso` sí, y con un solo coeficiente.

5.5. Sensibilidad al número de folds: ¿y si k no fuera 5?

Toda la sección anterior descansa sobre $k = 5$. Conviene preguntarse cuánto de la respuesta es del modelo y cuánto es de esa elección.

Qué se mide y qué no. Repetir la *selección completa* —las 19 configuraciones, para varios valores de k — cuesta horas por cada k : el Lasso de grado 4 sobre 1364 columnas domina el costo. Este trabajo no reporta esa corrida. Lo que sí se mide, y es la pregunta más directa, es el **barrido controlado**: se fija la configuración de producción y se varía sólo k , con lo cual la única causa de las diferencias es k y no un cambio de modelo. Los resultados están en `resultados/sensibilidad_k.csv` y son los de la figura 8.

Nada de esto vuelve a tocar el test. El módulo `src/sensibilidad_k.py` rehace el mismo split con la misma semilla y descarta `idx_test` sin usarlo: todo ocurre dentro de las 1070 filas de train.

Qué le pasa a los dos números que reporta este informe. Aparecen dos hallazgos distintos, y el primero corrige una lectura tentadora.

Sensibilidad al número de folds — configuración de producción fija (lineal grado 1, sin regularización)

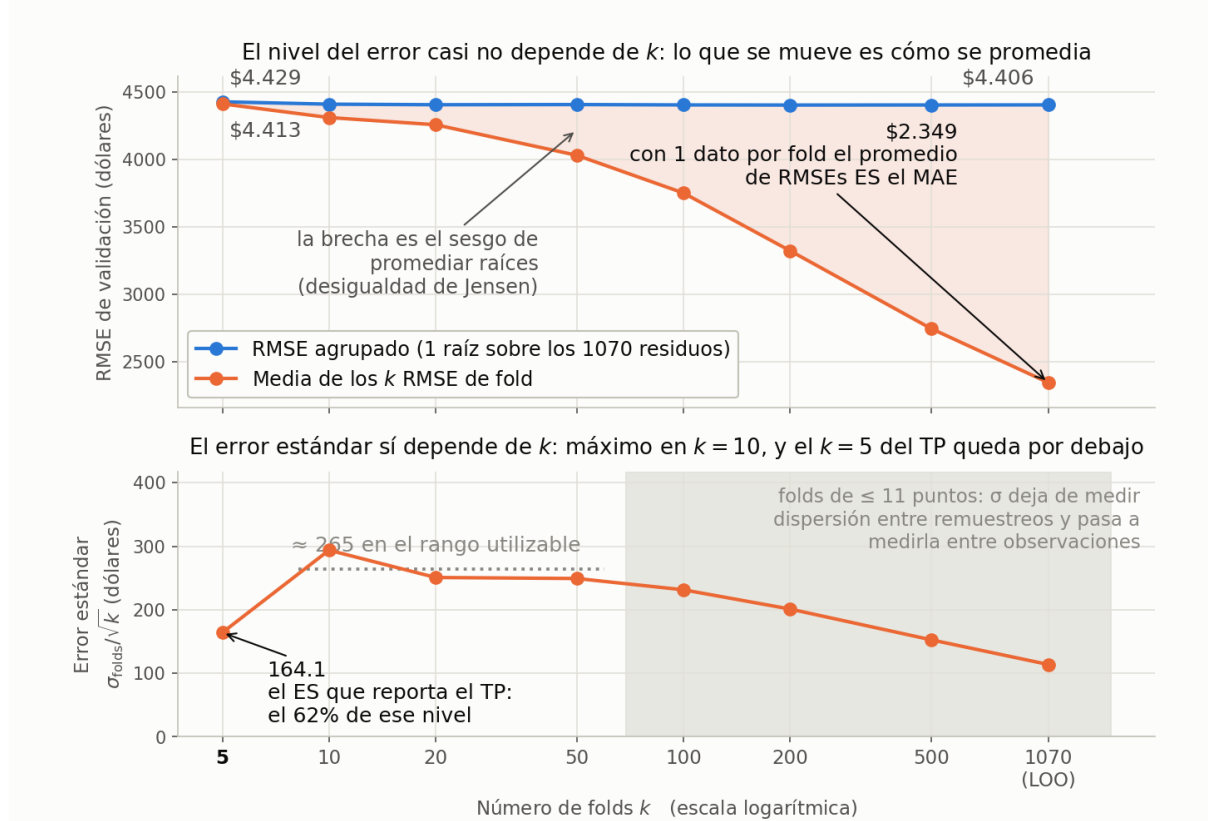


Figura 8: Sensibilidad al número de folds, con la configuración de producción fija (lineal grado 1, sin regularizar, once características). Los ocho valores de k dejan folds de 214, 107, 54, 21, 11, 5, 2 y 1 punto. **Arriba:** el nivel del error casi no depende de k —la curva agrupada va de 4428,7 a 4405,9, un 0,51 %—; lo que se desploma es la *forma de promediarlo*, y el área sombreada entre las dos curvas es ese sesgo. **Abajo:** el error estándar sí depende de k , con un máximo de 293,9 en $k = 10$ y una caída posterior; el $k = 5$ del informe vale 164,1. La banda gris marca dónde los folds son tan chicos que σ deja de medir dispersión entre remuestreos.

Primero: la caída del RMSE es un artefacto de la métrica, no del modelo. El pipeline reporta la *media de los k RMSE de fold*, y esa media baja 2064,9 dólares (4413,4 → 2348,5) al ir de $k = 5$ a LOO. Es tentador leerlo como que $k = 5$ es fuertemente pesimista, pero no lo es: la raíz es cóncava, así que por desigualdad de Jensen

$$\text{media}_i \sqrt{\text{ECM}_i} < \sqrt{\text{media}_i \text{ECM}_i}, \quad (4)$$

con una brecha que crece a medida que los folds se achican. Calculando en cambio el **RMSE agrupado** —una sola raíz sobre los 1070 residuos *out-of-fold* juntos, que no tiene ese sesgo— el error va de 4428,7 a 4405,9: **22,8 dólares, un 0,51 %**, y agotados ya en $k = 20$. El sesgo pesimista real de $k = 5$ es despreciable.

Ese mismo fenómeno, llevado al extremo, es lo que vuelve inservible a *leave-one-out* con este pipeline: con un solo dato por fold $\text{RMSE}_i = |y_i - \hat{y}_i|$, y el promedio de los k RMSE **es literalmente el MAE** (2348,5) —un 46,7 % por debajo del RMSE agrupado del mismo k (4405,9), y no comparable contra el RMSE de test de este informe—. LOO no es «más validación»: rompe la métrica en silencio, además de exigir ajustar cada configuración 1070 veces en lugar de 5.

Segundo: el error estándar tiene un único pico, en $k = 10$, y no una meseta. Midiendo la grilla densa — $k = 5, 10, 20, 50, 100, 200, 500, 1070$ — el ES sube de 164,1 en $k = 5$ a un máximo de 293,9 en $k = 10$, y de ahí cae de forma monótona: 250,9 / 249,3 / 231,6 / 201,3

/ 152,8 / 114,0 hasta LOO. No hay meseta entre $k = 10$ y $k = 50$: el pico está en un solo punto y la curva ya está bajando en $k = 20$.

El mecanismo se verifica contra los propios datos: si σ (el desvío del RMSE entre folds) creciera exactamente como $\sqrt{k}$, el ES sería constante. De $k = 5$ a 10, σ salta $2,53\times$ ($367,0 \rightarrow 929,5$) donde $\sqrt{k}$ predice sólo $1,41\times$ —la σ de $k = 5$ está estimada con *cinco* números y `ddof=0`, un estimador ruidoso y sesgado hacia abajo—, y ese exceso es lo que empuja el ES hacia arriba. Pasado el pico la relación se invierte: σ sigue creciendo, pero en promedio más lento que $\sqrt{k}$ (de $k = 500$ a LOO salta $1,09\times$ donde $\sqrt{k}$ predice $1,46\times$), porque con folds de 2 o 1 punto σ deja de medir variabilidad entre remuestreos y empieza a saturar contra la dispersión de los residuos individuales —la banda gris de la figura 8.

Las razones son dato medido; que las causas sean esas dos es la lectura más razonable del patrón, no una descomposición formal de la varianza. Lo que sí queda establecido es que el pico, 293,9 en $k = 10$, es $1,79$ veces el 164,1 que la regla de 1 ES usó en la selección del punto 5.

Consecuencia. No se cambia el esquema. El nivel del error no depende de k una vez corregida la forma de promediar, así que el RMSE de validación que reporta el punto 4 no es un artefacto de haber puesto 5. Lo que sí conviene declarar es la otra mitad: el error estándar que usa la regla de 1 error estándar depende bastante de k , y con $k = 10$ habría sido $1,79$ veces el 164,1 usado en el punto 5. Eso no debilita la elección del modelo simple —la refuerza: con un error estándar más ancho hay *más* configuraciones estadísticamente indistinguibles del mejor, no menos, y la más simple sigue siendo la misma.

5.6. Dónde vive el error que queda

El modelo de producción llega a 4.288,52 dólares de RMSE de test. La pregunta que sigue es si se puede bajar más, y con qué. La contesta un diagnóstico *out-of-fold* sobre train: para cada una de las 1070 filas, la predicción sale de un modelo que **no la vio** durante el ajuste —el mismo esquema de 5 folds del punto 4, pero guardando el residuo de cada fila en vez de sólo el RMSE agregado— en `src/diagnostico_residuos.py` (`resultados/diagnostico_residuos.csv`). No toca `X_test`: es un diagnóstico sobre train, no una evaluación más del modelo.

Tabla 8: Concentración del error cuadrático (SSE) sobre las filas de train, ordenadas de mayor a menor residuo.

Fracción de filas	Filas	% del SSE total
1 %	10	23,53 %
5 %	53	75,38 %
10 %	107	91,54 %

El error no está repartido. Diez personas —el 1 % del train— explican casi una cuarta parte del error cuadrático total. El 5 % explica tres cuartas partes. El modelo no está mal en todos lados por igual: está muy bien en la enorme mayoría de las filas y muy mal en un núcleo chico.

Tabla 9: Reparto del error cuadrático por población. El error se concentra en los **no fumadores**, no en los fumadores obesos que motivaron `fumador_obeso`.

Población	n	RMSE	% del SSE total
No fumadores	850	4615	86,3 %
Fumadores, $\text{bmi} \leq 30$	105	3407	5,8 %
Fumadores, $\text{bmi} > 30$	115	3804	7,9 %

Por población. Es un resultado que a primera vista sorprende: adentro de “no fumadores” hay, otra vez, una subpoblación de facto que ni `fumador_obeso` ni `bmi_si_fuma` distinguen, porque las dos características derivadas están construidas alrededor de `smoker=yes`.

El núcleo: 28 personas que ninguna columna distingue. Filtrando a los no fumadores con `charges > 25,000` —28 filas, 2,6 % del train— ese grupo aporta el **41,9 % del error cuadrático total**, y el modelo los **subestima en promedio 17.378 dólares**. Es, con amplio margen, el bloque más grande de error de todo el dataset.

Y no se los puede identificar con las columnas disponibles:

Tabla 10: Perfil de las 28 personas del núcleo contra el resto de los no fumadores. La única diferencia que se nota es la edad.

Variable	Estas 28 personas	Resto de los no fumadores
age (media)	50,79	39,11
bmi (media)	31,23	30,69
children (media)	1,39	1,10
sex	54 % F / 46 % M	51 % F / 49 % M
region	pareja a la del resto	pareja a la del resto

Bmi, `children`, sexo y región son prácticamente el mismo perfil que el resto de los no fumadores; sólo la edad se separa (50,8 contra 39,1), y no alcanza para aislar al grupo. Ninguna combinación de las columnas originales —ni siquiera sumando las tres características derivadas del pipeline vigente— lo separa del resto antes de ver `charges`.

Conclusión: error irreducible con este dataset. Ese ~ 42 % del error cuadrático **no es falta de capacidad del modelo**: falta la variable que explica el gasto —una patología, una cirugía, una condición preexistente— y esa variable no está en el CSV. Ninguna familia de modelos, por más flexible que sea, puede recuperar una señal que no está en las columnas de entrada; agregarle grados de libertad ahí sólo memoriza ruido, que es exactamente el sobreajuste que documenta el punto 4 en los grados 3 y 4.

El diagnóstico corre sobre el pipeline de once características, o sea con las tres derivadas ya incorporadas, y aun así ese núcleo concentra el 41,9 % del error: las columnas que resolvieron el problema de los fumadores no tocan este grupo, porque las tres están construidas alrededor de `smoker=yes`. **Para bajar ese tramo del error hace falta otra variable, no otro modelo.**

6. Limitaciones

- **Colinealidad en grados altos.** En grado 4, el 68,0 % de las 1364 columnas son redundantes (rango efectivo 436). Hay tres causas. Primera: una dummy binaria elevada a una potencia sigue teniendo sólo dos valores posibles, así que es una función afín exacta de la dummy original —por ejemplo $\text{smoker=yes}^2 = 1,456866 \cdot \text{smoker=yes} + 1,0$, con un

residuo de $1,5 \times 10^{-14}$, matemáticamente exacto salvo error de punto flotante—. Segunda: el producto de dos dummies del mismo one-hot cae dentro del espacio generado por las dummies individuales más la constante, porque 3 dummies y una constante ya generan todas las funciones posibles sobre las 4 regiones. Tercera: la expansión polinómica **duplica exactamente** a dos de las características derivadas a partir de grado 2 — age^2 coincide con `edad_al_cuadrado`, y $\text{bmi} \cdot \text{smoker}=\text{yes}$ con `bmi_si_fuma`—, lo que empuja la redundancia hasta el 68,0 % y explica que cuatro configuraciones Lasso no converjan. En grado 1, el de producción, no hay duplicación: las once columnas son de rango completo.

Conviene aclarar que ese producto **no es idénticamente cero**: lo sería en la codificación cruda 0/1, pero como el pipeline estandariza antes de expandir, cada dummy toma valores distintos de 0 y 1 y el producto también. Sigue siendo redundante, por la segunda razón, no por ser nulo. La consecuencia práctica es que **dentro de un grupo colineal el reparto de coeficientes es arbitrario**: no se pueden interpretar de a uno cuando el grado es alto.

- **Una sola partición train/test, con una sola semilla.** El RMSE de test reportado tiene varianza asociada a qué filas cayeron del lado de test, que este trabajo no midió. Repetir el split con otra semilla daría un número distinto, presumiblemente cercano pero no idéntico.
- **La partición de test se reutilizó a lo largo del desarrollo.** A medida que el conjunto de características se fue cerrando, el test se evaluó más de una vez sobre las mismas 267 filas. En todas las corridas la elección del modelo se hizo íntegramente con validación cruzada sobre `train` —ni `src/experimentos.py`, ni `src/evidencia_features.py`, ni `src/diagnostico_residuos.py` construyen `X_test`, y eso se verifica con un `grep`—, así que ninguna decisión ajustable miró el resultado. Aun así el costo es real: el protocolo ideal pide una partición nueva cada vez que cambia el modelo, y cada evaluación extra sobre la misma erosiona un poco su independencia.
- **El umbral de fumador_obeso no se validó como hiperparámetro.** Se tomó el de la OMS, que es externo al dataset. El barrido de umbrales lo confirma *a posteriori*, pero si el corte se hubiera elegido por ese barrido sería un hiperparámetro ajustado contra validación y el RMSE de validación quedaría sesgado a la baja. No es el caso, y la distinción importa.
- **El dataset es simulado.** Proviene de estadísticas demográficas —el origen es el dataset del libro de Lantz— y no del registro de una aseguradora real, así que las relaciones que el modelo aprendió reflejan cómo se construyó la simulación.
- **El RMSE penaliza el error al cuadrado**, con lo cual la métrica está dominada por los casos caros. Un modelo puede tener un RMSE relativamente bueno y aun así errar de forma proporcionalmente grande en los casos baratos.

7. Guion de la presentación (10 minutos)

Tabla 11: Reparto del tiempo. La introducción teórica y el punto 5 son lo que más se evalúa.

Min.	Qué se dice	Figura
0-1	Dataset (<code>insurance.csv</code> , 1338 filas) y la pregunta del TP: predecir <code>charges</code> y elegir un modelo defendible, no sólo el de menor error.	—
1-3	Introducción teórica: por qué <code>train</code> no alcanza, qué rol cumple cada conjunto, por qué el <code>test</code> se toca una sola vez, y qué gana la validación cruzada frente a un único <code>split</code> .	—
3-4	Punto 1: dónde se parte el dataset y por qué el EDA va después; por qué se conservan los 115 outliers (97,4 % fumadores) y por qué IQR y no <code>z-score</code> .	04-outliers
4-5	Puntos 2 y 3: el pipeline y por qué la regresión polinómica sigue siendo lineal en los parámetros.	—
5-7	Punto 4: las curvas por grado, dónde aparece el sobreajuste (la brecha de 61,8 a 84.537,9, el desvío de ± 367 a ± 47.000) y cómo Lasso lo corrige.	01-curvas, 02-camino
7-9	Punto 5, el núcleo: por qué el lineal grado 1 gana la CV siendo además el más simple del espacio, por qué 7 de las 15 son indistinguibles y confirman la misma elección, y el RMSE prometido ($R^2 = 0,871$), con el sesgo del mínimo.	una-es
9-10	Limitaciones y cierre.	—

A. Reproducibilidad

El repositorio corre con Python 3.10 o superior y solamente `numpy`, `pandas` y `matplotlib`. No usa `scikit-learn` ni `pytest`.

Comando	Qué hace
<code>python -m src.datos</code>	Punto 1: análisis exploratorio
<code>python -m src.experimentos</code>	Puntos 2 a 5 (unos 27 minutos)
<code>python -m src.evidencia_features</code>	La evidencia numérica de las tres derivadas, de la estratificación y de <code>children</code>
<code>python -m src.diagnostico_residuos</code>	El diagnóstico de residuos y el error irreducible
<code>python -m src.graficos</code>	Genera las figuras de este informe
<code>python -m src.tablas</code>	Regenera las tablas del punto 4 desde los CSV
<code>python -m src.sensibilidad_k</code>	Barrido de k de la sección 5.5
<code>python -m tests.test_validacion</code>	Suite del módulo de partición y métricas
<code>python -m tests.test_preproceso</code>	Suite de codificación, escalado y expansión
<code>python -m tests.test_modelos</code>	Suite de los modelos
<code>python -m tests.test_paleta</code>	La paleta de las figuras, bajo simulación de daltonismo

Los resultados están versionados en `resultados/` (`cv_lineal.csv`, `cv_lasso.csv`, `modelo_elegido.json`, `evaluacion_test.json`, `evidencia_features.csv`, `diagnostico_residuos.csv` y `sensibilidad_k.csv`), junto con la salida completa de la corrida en `informe/salida-seleccion.txt`, de modo que no hace falta correr nada para verificar los números de este informe.

Procedencia del dataset. El archivo se descargó del enlace que da el enunciado ([kaggle.com/datasets/ mirichoi0218/insurance](https://kaggle.com/datasets/mirichoi0218/insurance)) y se verificó que la copia usada es byte a byte idéntica al original publicado:

SHA256 = 388eff679557d08ac19f463d025de5e0b4adc482537c8456d19934d78621fd47